Project 6:- Maven Selenium Project – MyMavenSeleniumApp04

Objective

This project involves writing a Selenium script that operates across three different websites in separate browser tabs, automating login forms, a product search, and user authentication all within a single test run.

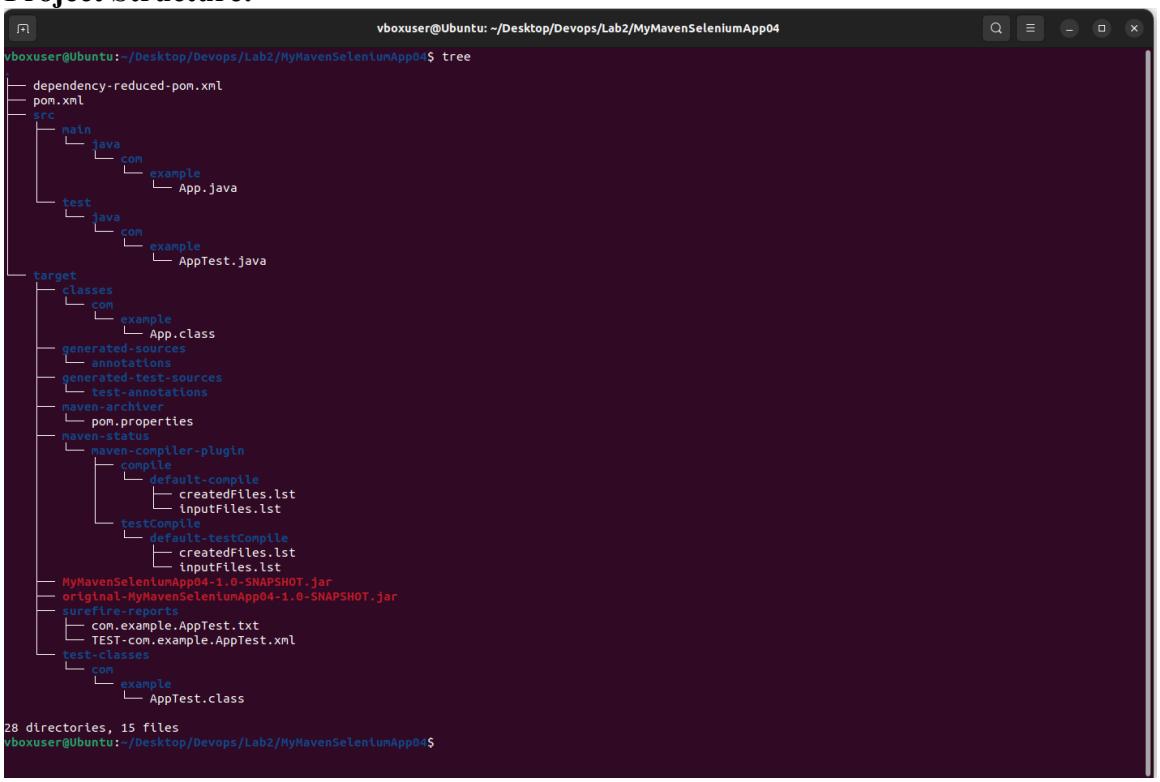
Websites Used for Automation

1. https://www.saucedemo.com/
2. https://automationexercise.com/products
3. https://practicetestautomation.com/practice-test-login/

Step 1: Create a Maven Project Using Terminal

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp04
  -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Project Structure:



Step 2: Add Selenium Dependency in pom.xml

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.41.0</version>
</dependency>
```

Step 3: Inspect Elements in the Websites

Inspect each site using DevTools and note the following HTML element IDs that the script will reference:
SauceDemo: user-name, password, login-button
Automation Exercise: search_product, submit_search

Practice Test Automation: username, password, submit

Step 4: Create App.java

Open App.java at src/main/java/com/example/ and replace the contents with the code below:

```
package com.example;


import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.WindowType;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;

public class App {
  public static void main(String[] args) throws InterruptedException {
    ChromeOptions options = new ChromeOptions();
    options.addArguments("--start-maximized");
    WebDriver driver = new ChromeDriver(options);
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

    driver.get("https://www.saucedemo.com/");
    driver.findElement(By.id("user-name")).sendKeys("standard_user");
    driver.findElement(By.id("password")).sendKeys("secret_sauce");
    driver.findElement(By.id("login-button")).click();

    driver.switchTo().newWindow(WindowType.TAB);
    driver.get("https://automationexercise.com/products");
    driver.findElement(By.id("search_product")).sendKeys("Men Tshirt");
    driver.findElement(By.id("submit_search")).click();
    WebElement product = wait.until(
      ExpectedConditions.visibilityOfElementLocated(
        By.cssSelector("a[data-product-id='2']")));
    product.click();

    driver.switchTo().newWindow(WindowType.TAB);
    driver.get("https://practicetestautomation.com/practice-test-login/");
    driver.findElement(By.id("username")).sendKeys("student");
    driver.findElement(By.id("password")).sendKeys("Password123");
    driver.findElement(By.id("submit")).click();
```

```
        Thread.sleep(5000);
        driver.quit();
    }
}
```

Step 5: Build the Maven Project

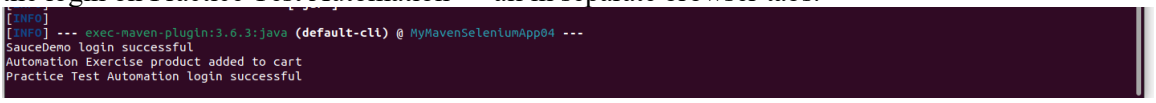
```
mvn clean compile
```

Step 6: Run the Application

```
mvn exec:java
```

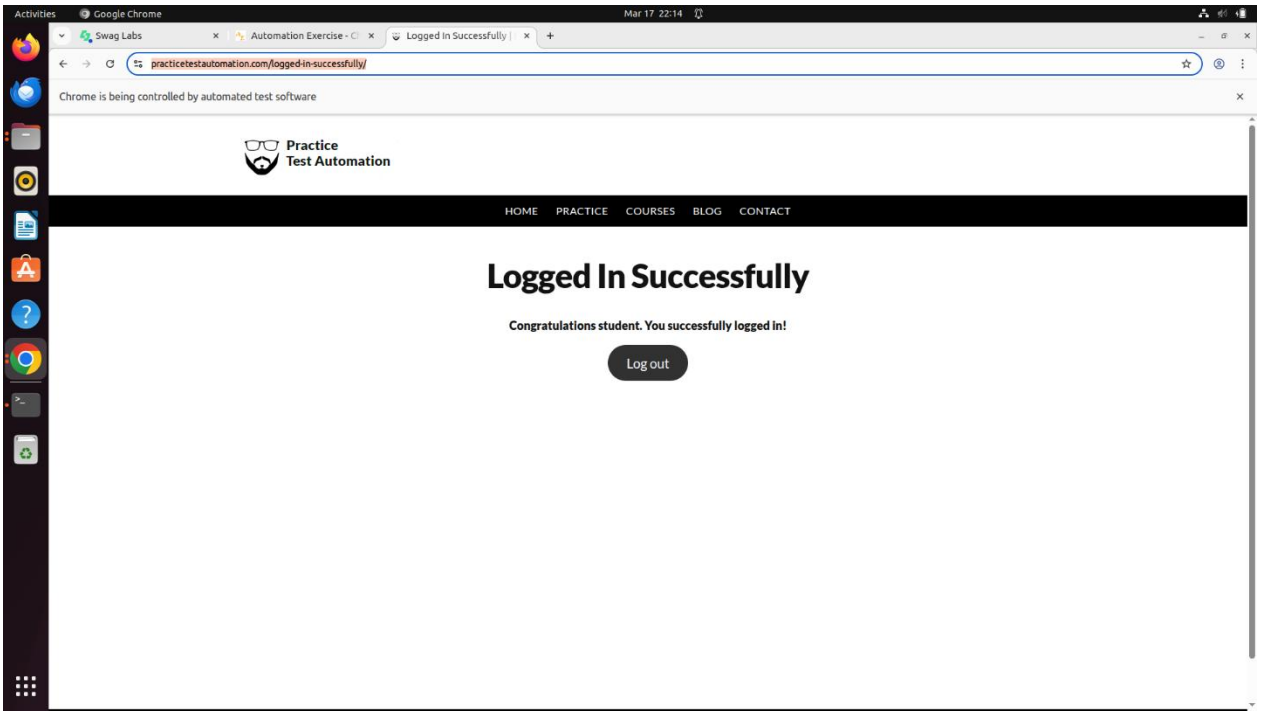
Output

The script successfully logs in to SauceDemo, performs a product search on Automation Exercise, and completes the login on Practice Test Automation — all in separate browser tabs.



Conclusion

Working across multiple sites in a single session proved that Selenium WebDriver is well-suited for complex, multi-step automation. Tab management and explicit waits made the script stable and reliable.



Jenkins Pipeline for MyMavenSeleniumApp04

Step 1: Open the Jenkins Dashboard in order to create a new item.

Step 2: Click on New Item in order to create a New Item. Give the name as "MyMavenSeleniumApp04" and Item type as Pipeline, then click Create.

Step 3: Configure the settings by defining the Definition as "Pipeline Script from SCM" and SCM as "Git". Paste the Repository URL of the project from GitHub and choose Credentials as "GitHub Credentials" which was created earlier.

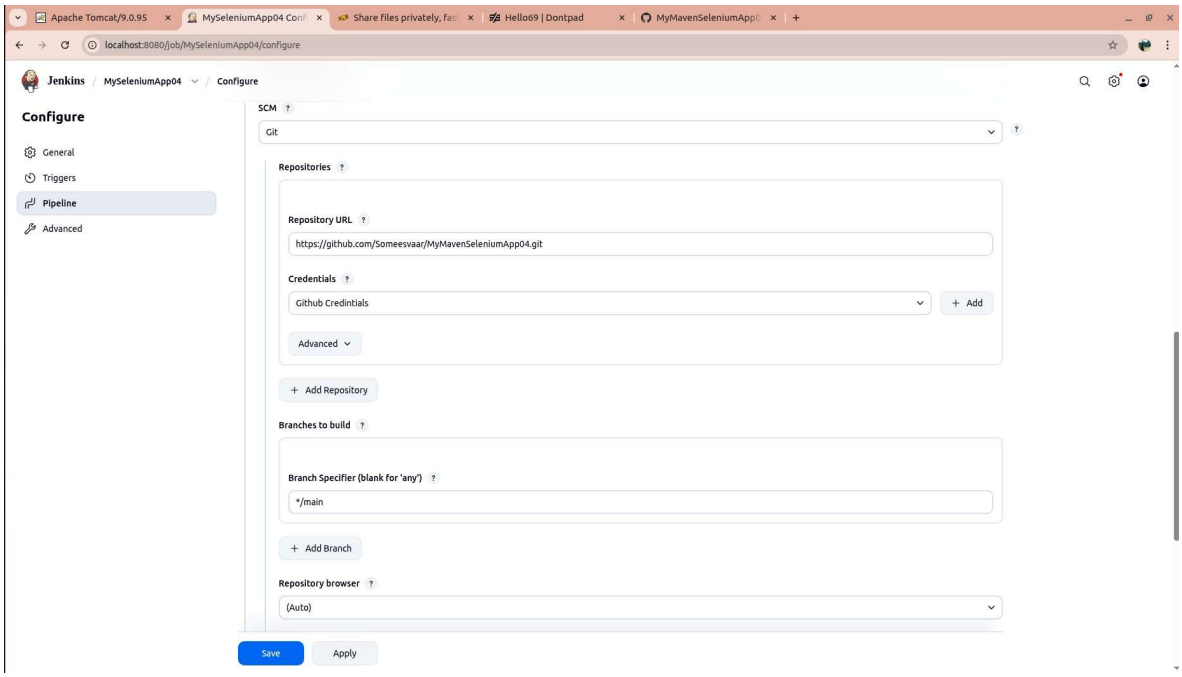


Fig D-1: Jenkins Pipeline configuration – Repository URL for MyMavenSeleniumApp04.git with GitHub Credentials

Step 4: In Terminal, go to the directory created for MyMavenSeleniumApp04 and create a new file called "Jenkinsfile".

Step 5: Paste the following code inside the Jenkinsfile:

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url:
'https://github.com/Someesvaar/MyMavenSeleniumApp04.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }
```

```
        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }




        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenSeleniumApp04-1.0-SNAPSHOT.jar'
            }
        }



    }
     post {
        success {
            echo 'Build and deployment successful!'
        }
        failure {
            echo 'Build failed!'
      }
     }
}
```

And add the following to App.java
options.addArguments("--headless=new"); // modern headless mode
options.addArguments("--no-sandbox"); // required in Jenkins/Linux
options.addArguments("--disable-dev-shm-usage"); // prevents crashes
options.addArguments("--disable-gpu");

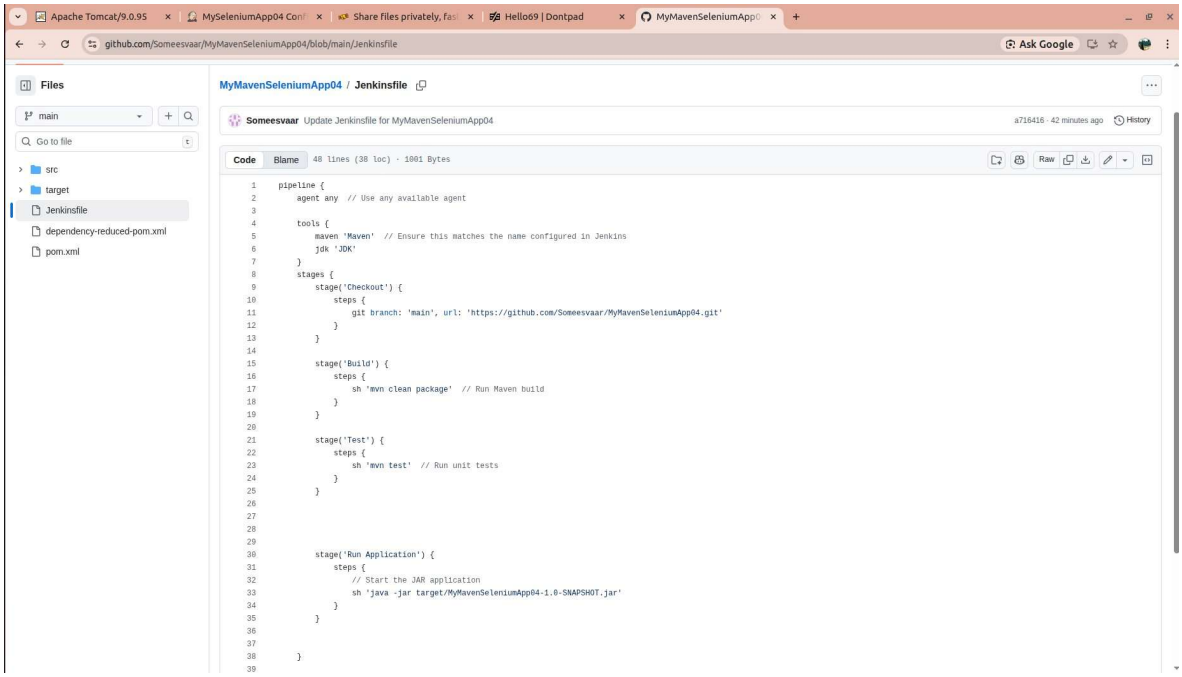


Fig D-2: Jenkinsfile committed to the MyMavenSeleniumApp04 GitHub repository

Step 6: Update pom.xml to change the versions of source and target from 1.7 to 1.8.

Step 7: Push the changes and commit by pushing the "Jenkinsfile" to the respective repository on GitHub.

Step 8: Go to Jenkins and Build the Pipeline, then check the Stage View to confirm successful execution.

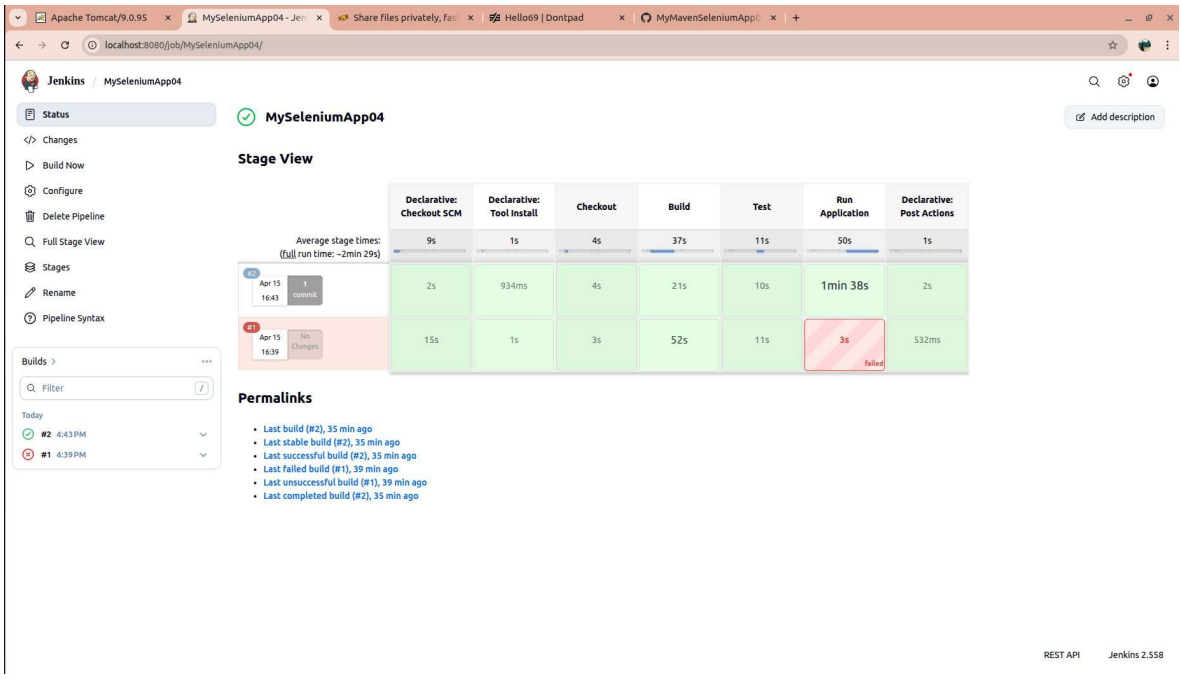


Fig D-3: Jenkins Stage View showing Build #2 successful for MyMavenSeleniumApp04 (~2min 29s total)